

# Predicting Food Trends with Google Search Data

Parsa Vetry

2026-08-15

## Introduction

Food trends move quickly, a dish can go from niche to viral in the span of a month or two. This project asks the question: can Google search interest data anticipate which foods are about to grow in popularity?

To explore this question, I pulled six years of Google Search Trends data from 2019 to 2026 for about 200 food and beverages. From each keyword's search data, I engineered a set of features meant to predict whether or not a food trend will grow.

My first approach was to predict the exact search-interest level six months into the future, which turned out to be the wrong framing. My strongest model achieved an R-squared of about 0.74, while a naive model assuming that nothing changes achieved an R-squared value of .66; search interest is highly persistent month to month.

I then reframed my problem as a binary question: will a keyword's search interest be higher, on average, six months from now than it is today? My direction based approach carried real signal, as opposed to the level-prediction approach. My best model achieved a strong AUC of .86, as opposed to the naive baseline of 0.5.

## Data Collection

```
library(gtrendsR)
```

```
## Warning: package 'gtrendsR' was built under R version 4.5.3
```

```
library(ggplot2)
library(lubridate)
```

```
## Warning: package 'lubridate' was built under R version 4.5.2
```

```
##
```

```
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      date, intersect, setdiff, union
```

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(tidyr)  
library(zoo)
```

```
## Warning: package 'zoo' was built under R version 4.5.3  
  
##  
## Attaching package: 'zoo'  
  
## The following objects are masked from 'package:base':  
##  
##   as.Date, as.Date.numeric
```

```
keywords <- c("matcha", "oat milk", "kombucha", "boba", "birria", "tacos", "empanadas", "tahini", "hot l
```

Create dataset from keywords

```
dir.create("trends_raw_cache", showWarnings = FALSE)  
  
get_trend_raw <- function(kw) {  
  
  file <- paste0("trends_raw_cache/", gsub(" ", "_", kw), ".rds")  
  
  if (file.exists(file)) {  
    return(readRDS(file))  
  }  
  
  Sys.sleep(runif(1, 155, 205))  
  
  result <- tryCatch({  
  
    gtrends(  
      kw,  
      geo = "US",  
      time = "2019-01-01 2026-08-14"  
    )$interest_over_time %>%  
    mutate(  
      date = as.Date(date),
```

```

      hits = as.numeric(gsub("<1", "0.5", hits))
    ) %>%
    select(date, hits) %>%
    mutate(keyword = kw)

  }, error = function(e) {
    message("Failed: ", kw)
    NULL
  })

  if (!is.null(result)) {
    saveRDS(result, file)
  }

  result
}

all_trends_raw <- bind_rows(lapply(keywords, get_trend_raw))

saveRDS(all_trends_raw, "food_trends_raw.rds")

all_trends_raw <- readRDS("food_trends_raw.rds")

```

## Feature Engineering

```

epsilon <- 0.01

features <- all_trends_raw %>%
  group_by(keyword) %>%
  arrange(date) %>%
  mutate(
    growth_3m = ifelse(lag(hits, 3) == 0, 0, (hits / lag(hits, 3)) - 1),
    growth_6m = ifelse(lag(hits, 6) == 0, 0, (hits / lag(hits, 6)) - 1),
    growth_12m = ifelse(lag(hits, 12) == 0, 0, (hits / lag(hits, 12)) - 1),
    mean_3m = rollapply(hits, 3, mean, fill = NA, align = "right"),
    mean_6m = rollapply(hits, 6, mean, fill = NA, align = "right"),
    mean_12m = rollapply(hits, 12, mean, fill = NA, align = "right"),
    mean_ratio_3m_12m = mean_3m / (mean_12m + epsilon),
    slope_3m = rollapply(hits, 3, function(x) coef(lm(x ~ seq_along(x)))[2], fill = NA, align = "right"),
    slope_6m = rollapply(hits, 6, function(x) coef(lm(x ~ seq_along(x)))[2], fill = NA, align = "right"),
    slope_12m = rollapply(hits, 12, function(x) coef(lm(x ~ seq_along(x)))[2], fill = NA, align = "right"),
    acceleration_3m = slope_3m - lag(slope_3m),
    acceleration_6m = slope_6m - lag(slope_6m),
    acceleration_12m = slope_12m - lag(slope_12m),
    volatility_3m = rollapply(hits, 3, sd, fill = NA, align = "right"),
    volatility_6m = rollapply(hits, 6, sd, fill = NA, align = "right"),
    volatility_12m = rollapply(hits, 12, sd, fill = NA, align = "right"),
    future_mean_6m = rollapply(lead(hits, 1), 6, mean, fill = NA, align = "left"),
    future_mean_12m = rollapply(lead(hits, 1), 12, mean, fill = NA, align = "left"),
    future_growth_6m = (future_mean_6m + epsilon) / (mean_6m + epsilon) - 1,

```

```

future_6m = lead(hits, 6),
max_12m = rollapply(hits, 12, max, align = "right", fill = NA, na.rm = TRUE),
breakout_12m = ifelse(hits > lag(max_12m), 1, 0),
drawdown = hits / max_12m - 1,
monthly_change = hits - lag(hits),
trend_strength = abs(slope_12m) / (volatility_12m + epsilon),
direction = factor(ifelse(future_mean_6m > mean_6m, 1, 0), levels = c(0,1))
)%>%
ungroup() %>%
filter(
  !is.na(growth_3m),
  !is.na(growth_6m),
  !is.na(growth_12m),
  !is.na(mean_3m),
  !is.na(mean_6m),
  !is.na(mean_12m),
  !is.na(mean_ratio_3m_12m),
  !is.na(slope_3m),
  !is.na(slope_6m),
  !is.na(slope_12m),
  !is.na(acceleration_3m),
  !is.na(acceleration_6m),
  !is.na(acceleration_12m),
  !is.na(volatility_3m),
  !is.na(volatility_6m),
  !is.na(volatility_12m),
  !is.na(future_mean_6m),
  !is.na(future_mean_12m),
  !is.na(future_growth_6m),
  !is.na(future_6m),
  !is.na(max_12m),
  !is.na(breakout_12m),
  !is.na(drawdown),
  !is.na(monthly_change),
  !is.na(trend_strength),
  !is.na(direction)
)

```

## Predicting Future Search Volume

We will split by time to avoid look-ahead bias

```

# Split the training and test data
set.seed(123)

cutoff_date <- as.Date("2025-01-01")
test_end <- as.Date("2026-06-01")

train <- features %>% filter(date < cutoff_date)
test <- features %>% filter(date >= cutoff_date & date < test_end)

```

## Ranger

```
library(ranger)
```

```
## Warning: package 'ranger' was built under R version 4.5.3
```

```
library(mlr3)
```

```
## Warning: package 'mlr3' was built under R version 4.5.3
```

```
library(mlr3learners)
```

```
## Warning: package 'mlr3learners' was built under R version 4.5.3
```

```
features_to_use <- c(
  "growth_12m",
  "mean_ratio_3m_12m",
  "acceleration_3m",
  "acceleration_12m",
  "volatility_3m",
  "volatility_12m",
  "drawdown",
  "monthly_change",
  "trend_strength",
  "acceleration_6m",
  "volatility_6m"
)

task <- as_task_regr(train, target = "future_mean_6m", id = "all_trends_raw")

task$select(features_to_use)

learner_ranger <- lrn("regr.ranger", importance = "impurity")

learner_ranger$train(task)

learner_ranger$importance()
```

```
##      volatility_12m      drawdown      volatility_6m mean_ratio_3m_12m
##      1430901.2      789616.4      767160.4      554975.0
## acceleration_12m      volatility_3m      growth_12m      acceleration_6m
##      340680.6      339410.9      253134.7      222815.3
##      trend_strength      acceleration_3m      monthly_change
##      178983.9      165964.6      108153.7
```

```
prediction_ranger <- learner_ranger$predict_newdata(test)
```

```

current_level <- test$hits

results_ranger <- test %>%
  select(keyword, date, hits, future_mean_6m) %>%
  mutate(
    predicted_6m = prediction_ranger$response,
    predicted_growth = (((predicted_6m + epsilon) / (current_level + epsilon)) - 1) * 100,
    actual_growth = ifelse(hits <= epsilon, NA, (future_mean_6m / hits) - 1),
    actual_growth_pct = actual_growth * 100,
    error = predicted_6m - future_mean_6m,
    absolute_error = abs(error),
    percent_error = ifelse(future_mean_6m <= epsilon, NA, (predicted_6m - future_mean_6m) / future_mean_6m)
  )

head(results_ranger)

```

```

## # A tibble: 6 x 11
##   keyword  date      hits future_mean_6m predicted_6m predicted_growth
##   <chr>   <date>    <dbl>         <dbl>         <dbl>         <dbl>
## 1 matcha  2025-01-01    55           75.7           48.8          -11.3
## 2 oat milk 2025-01-01    78           67.8           63.7          -18.3
## 3 kombucha 2025-01-01    70           63.5           66.0           -5.69
## 4 boba     2025-01-01    28           32.3           37.7           34.5
## 5 birria   2025-01-01    73           72.2           65.8           -9.92
## 6 tacos    2025-01-01    69           79.2           70.1            1.56
## # i 5 more variables: actual_growth <dbl>, actual_growth_pct <dbl>,
## #   error <dbl>, absolute_error <dbl>, percent_error <dbl>

```

```

mean(results_ranger$absolute_error)

```

```

## [1] 7.951764

```

## XGboost

```

library(xgboost)

```

```

## Warning: package 'xgboost' was built under R version 4.5.3

```

```

library(mlr3)
library(mlr3learners)

features_to_use <- c(
  "growth_12m",
  "mean_ratio_3m_12m",
  "acceleration_3m",
  "acceleration_12m",
  "volatility_3m",
  "volatility_12m",
  "drawdown",

```

```

"monthly_change",
"trend_strength",
"acceleration_6m",
"volatility_6m"
)

task <- as_task_regr(train, target = "future_mean_6m", id = "all_trends_raw")

task$select(features_to_use)

learner_xgboost <- lrn("regr.xgboost")

learner_xgboost$train(task)

prediction_xgboost <- learner_xgboost$predict_newdata(test)

learner_xgboost$importance()

```

```

##      volatility_12m      drawdown mean_ratio_3m_12m acceleration_12m
##      0.42301165      0.21943795      0.12465029      0.07057820
##      volatility_6m      growth_12m      trend_strength      acceleration_6m
##      0.04285780      0.03331002      0.02347833      0.01926930
##      acceleration_3m      volatility_3m      monthly_change
##      0.01913273      0.01351575      0.01075798

```

```

prediction_xgboost$score(msrs(c("regr.rmse", "regr.mae", "regr.rsq")))

```

```

## regr.rmse regr.mae regr.rsq
## 9.2569822 7.1215560 0.7346881

```

```

current_level <- test$hits

```

```

results_xgboost <- test %>%
  select(keyword, date, hits, future_mean_6m) %>%
  mutate(
    predicted_6m = prediction_xgboost$response,
    predicted_growth = (((predicted_6m + epsilon) / (current_level + epsilon)) - 1) * 100,
    actual_growth = ifelse(hits <= epsilon, NA, (future_mean_6m / hits) - 1),
    actual_growth_pct = actual_growth * 100,
    error = predicted_6m - future_mean_6m,
    absolute_error = abs(error),
    percent_error = ifelse(future_mean_6m <= epsilon, NA, (predicted_6m - future_mean_6m) / future_mean_6m)
  )

```

```

head(results_xgboost)

```

```

## # A tibble: 6 x 11
##   keyword  date      hits future_mean_6m predicted_6m predicted_growth
##   <chr>   <date>   <dbl>      <dbl>      <dbl>      <dbl>
## 1 matcha 2025-01-01    55        75.7        37.5        -31.8
## 2 oat milk 2025-01-01    78        67.8        63.1        -19.1

```

```
## 3 kombucha 2025-01-01 70 63.5 69.7 -0.393
## 4 boba 2025-01-01 28 32.3 36.7 30.9
## 5 birria 2025-01-01 73 72.2 70.7 -3.20
## 6 tacos 2025-01-01 69 79.2 74.0 7.31
## # i 5 more variables: actual_growth <dbl>, actual_growth_pct <dbl>,
## # error <dbl>, absolute_error <dbl>, percent_error <dbl>
```

```
mean(results_xgboost$absolute_error)
```

```
## [1] 7.121556
```

## Establishing a Naive Baseline + Model Comparison

```
library(mlr3benchmark)
```

```
## Warning: package 'mlr3benchmark' was built under R version 4.5.3
```

```
library(mlr3extralearners)
library(mlr3viz)
```

```
## Warning: package 'mlr3viz' was built under R version 4.5.3
```

```
features_to_use <- c(
  "growth_12m",
  "mean_ratio_3m_12m",
  "acceleration_3m",
  "acceleration_12m",
  "volatility_3m",
  "volatility_12m",
  "drawdown",
  "monthly_change",
  "trend_strength",
  "acceleration_6m",
  "volatility_6m"
)

task <- as_task_regr(features, target = "future_mean_6m", id = "all_trends_raw")
task$select(features_to_use)

train_ids <- which(features$date < cutoff_date)
test_ids <- which(features$date >= cutoff_date & features$date < test_end)

resampling <- rsmp("custom")
resampling$instantiate(
  task,
  train_sets = list(train_ids),
  test_sets = list(test_ids)
)
```

```

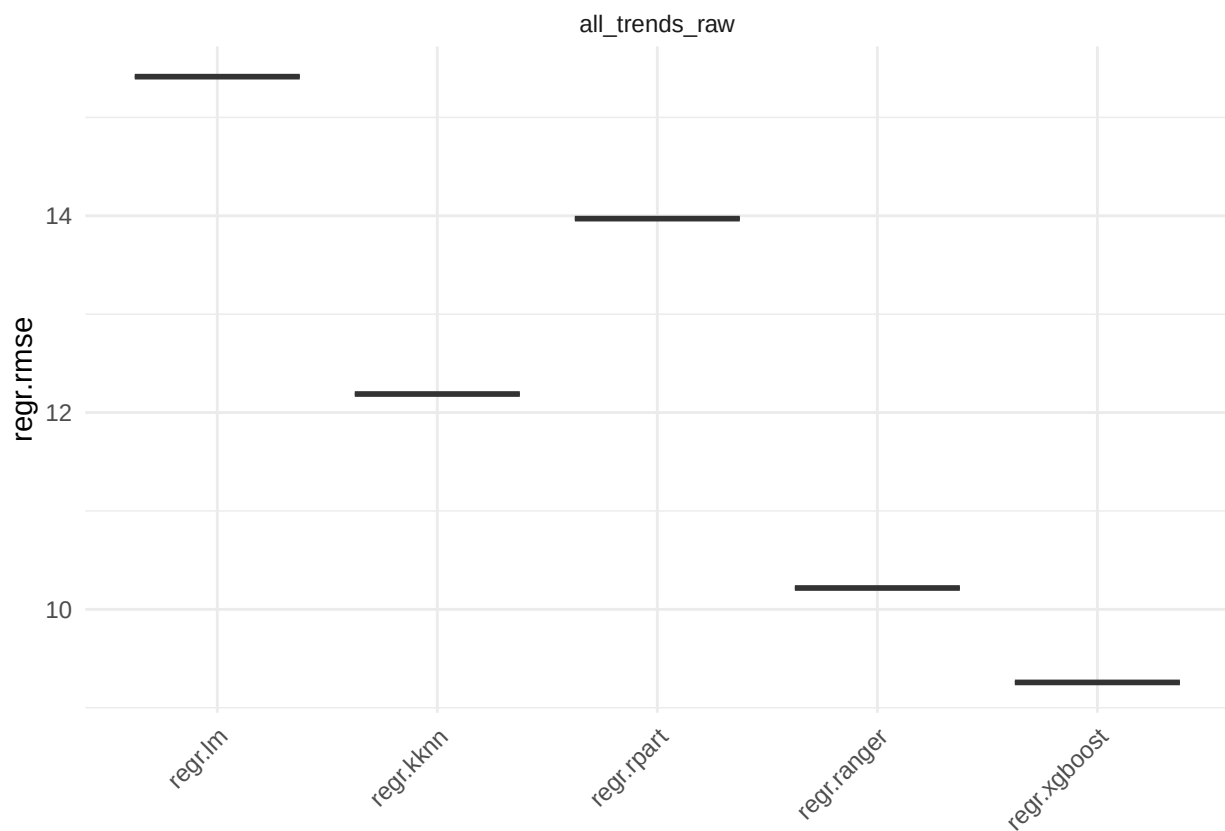
learners = lrns(c("regr.lm", "regr.kknn", "regr.rpart", "regr.ranger", "regr.xgboost"))
bmr_design <- benchmark_grid(tasks = task, learners = learners, resamplings = resampling)
bmr <- benchmark(design = bmr_design)

```

```
## Warning: package 'future' was built under R version 4.5.2
```

```
## Warning: package 'kknn' was built under R version 4.5.3
```

```
autoplot(bmr, measure = msr("regr.rmse"))
```



Create table with models and their corresponding measures of RMSE, MAE, and R-squared

```
bmr_table <- bmr$aggregate(measures = list(msr("regr.rmse"), msr("regr.mae"), msr("regr.rsq"))) %>% as.data.frame()
```

```

naive_rsqr <- function(actual, naive_pred) {
  1 - sum((actual - naive_pred)^2) / sum((actual - mean(actual))^2)
}

```

```

naive_pred <- test$mean_6m
naive_actual <- test$future_mean_6m

```

```

naive_rmse <- sqrt(mean((naive_actual - naive_pred)^2))
naive_mae <- mean(abs(naive_actual - naive_pred))
naive_r2 <- naive_rsqa(naive_actual, naive_pred)

naive_results <- data.frame(
  learner_id = "naive_no_change",
  regr.rmse = naive_rmse,
  regr.mae = naive_mae,
  regr.rsqa = naive_r2
)

full_comparison <- bind_rows(bmr_table, naive_results) %>%
  arrange(desc(regr.rsqa))

full_comparison

```

```

##      learner_id regr.rmse  regr.mae  regr.rsqa
## 1  regr.xgboost  9.256982  7.121556  0.7346881
## 2  regr.ranger  10.216853  7.959600  0.6768143
## 3 naive_no_change 10.447862  7.652625  0.6620343
## 4  regr.kknn  12.188311  9.482845  0.5400561
## 5  regr.rpart  13.971042  11.359549  0.3956683
## 6    regr.lm  15.414145  12.451209  0.2643747

```

## Level Prediction Falls Short

Against the naive “no change” baseline ( $rsq = 0.662$ ), the results are mixed. Only XGboost ( $rsq = 0.735$ ) clearly outperforms naive persistence, only adding about 0.07  $rsq$ . This finding motivates the shift to a different framing of the problem in the next section. Predicting the direction of change (up or down) turned out to reveal substantially more consistent signal across nearly every model.

## Predicting The Direction Of Change

Again, we will split by time to avoid look-ahead bias

```

# Split the training and test data
set.seed(123)

cutoff_date <- as.Date("2025-01-01")
test_end <- as.Date("2026-06-01")

train <- features %>% filter(date < cutoff_date)
test <- features %>% filter(date >= cutoff_date & date < test_end)

```

## Ranger

```

library(ranger)
library(mlr3)
library(mlr3learners)

features_to_use <- c(
  "growth_12m",
  "mean_ratio_3m_12m",
  "acceleration_3m",
  "acceleration_12m",
  "volatility_3m",
  "volatility_12m",
  "drawdown",
  "monthly_change",
  "trend_strength",
  "acceleration_6m",
  "volatility_6m"
)

task <- as_task_classif(train, target = "direction", id = "all_trends_raw")

task$select(features_to_use)

learner_ranger <- lrn("classif.ranger", predict_type = "prob", importance = "impurity")

learner_ranger$train(task)

learner_ranger$importance()

##      acceleration_6m mean_ratio_3m_12m      volatility_6m acceleration_12m
##      1378.5782      590.4968      584.5361      510.7610
##      growth_12m      trend_strength      volatility_12m      drawdown
##      503.2092      482.6963      392.3316      362.3429
##      acceleration_3m      monthly_change      volatility_3m
##      354.7996      344.0148      263.2444

prediction_ranger <- learner_ranger$predict_newdata(test)

results_direction <- test %>%
  select(keyword, date, hits, mean_6m, future_mean_6m, direction) %>%
  mutate(
    predicted_direction = prediction_ranger$response,
    prob_up = prediction_ranger$prob[, "1"],
    correct = predicted_direction == direction,
    actual_growth_pct = ifelse(hits <= epsilon, NA, ((future_mean_6m / hits) - 1) * 100)
  ) %>%
  arrange(desc(prob_up))

head(results_direction)

## # A tibble: 6 x 10
##   keyword  date      hits mean_6m future_mean_6m direction predicted_direction
##   <chr>    <date>    <dbl>  <dbl>         <dbl> <fct>      <fct>

```

```
## 1 elote      2025-03-01    46   41.3           60.2 1          1
## 2 pumpkin~  2025-08-01     6    2.83          24.7 1          1
## 3 hot dog   2025-03-01    40   34.3           57.3 1          1
## 4 zucchin~ 2025-04-01     8    8.17           32    1          1
## 5 hot dog   2025-02-01    35   34.3           57    1          1
## 6 bread p~  2025-08-01    39   41            58.3 1          1
## # i 3 more variables: prob_up <dbl>, correct <lgl>, actual_growth_pct <dbl>
```

```
prediction_ranger$score(msrs(c("classif.acc", "classif.bacc", "classif.auc", "classif.precision", "classif.recall", "classif.bbrier")))
```

```
##      classif.acc      classif.bacc      classif.auc      classif.precision
##      0.7582599      0.7685036      0.8611887      0.6361502
##      classif.recall      classif.bbrier
##      0.8077496      0.1595386
```

## XGboost

```
library(xgboost)
library(mlr3)
library(mlr3learners)

features_to_use <- c(
  "growth_12m",
  "mean_ratio_3m_12m",
  "acceleration_3m",
  "acceleration_12m",
  "volatility_3m",
  "volatility_6m",
  "volatility_12m",
  "drawdown",
  "monthly_change",
  "trend_strength",
  "acceleration_6m"
)

task <- as_task_classif(train, target = "direction", id = "all_trends_raw")

task$select(features_to_use)

learner_xgboost <- lrn("classif.xgboost", predict_type = "prob")

learner_xgboost$train(task)

prediction_xgboost <- learner_xgboost$predict_newdata(test)

learner_xgboost$importance()
```

```
##      acceleration_6m mean_ratio_3m_12m      growth_12m      trend_strength
##      0.25089812      0.11942582      0.09628189      0.09520984
##      volatility_6m      acceleration_12m      volatility_12m      drawdown
```

```
##          0.08684400          0.08645539          0.08095352          0.07200246
##    volatility_3m    acceleration_3m    monthly_change
##          0.04468469          0.04109122          0.02615305
```

```
prediction_xgboost$score(msrs(c("classif.acc", "classif.bacc", "classif.auc", "classif.precision", "classif.recall", "classif.bbrier")))
```

```
##          classif.acc          classif.bacc          classif.auc    classif.precision
##          0.7422907          0.7555314          0.8364209          0.6154721
##    classif.recall    classif.bbrier
##          0.8062593          0.2038128
```

```
prediction_direction <- learner_xgboost$predict_newdata(test)
```

```
results_direction <- test %>%
  select(keyword, date, hits, mean_6m, future_mean_6m, direction) %>%
  mutate(
    predicted_direction = prediction_direction$response,
    prob_up = prediction_direction$prob[, "1"],
    correct = predicted_direction == direction,
    actual_growth_pct = ifelse(hits <= epsilon, NA, ((future_mean_6m / hits) - 1) * 100)
  ) %>%
  arrange(desc(prob_up))

head(results_direction)
```

```
## # A tibble: 6 x 10
##   keyword date      hits mean_6m future_mean_6m direction predicted_direction
##   <chr>   <date>   <dbl>   <dbl>         <dbl> <fct>         <fct>
## 1 caprese 2025-03-01    27    25.5           42  1             1
## 2 elote   2025-02-01    43    41.8          59.3 1             1
## 3 elote   2025-03-01    46    41.3          60.2 1             1
## 4 caprese 2025-02-01    26    26.7          40.7 1             1
## 5 falafel 2025-01-01    52    42.8          49.8 1             1
## 6 hot dog 2025-02-01    35    34.3          57  1             1
## # i 3 more variables: prob_up <dbl>, correct <lgl>, actual_growth_pct <dbl>
```

## Direction Classification: Model Comparison

```
library(mlr3benchmark)
library(mlr3extralearners)
library(mlr3viz)
```

```
features_to_use <- c(
  "growth_12m",
  "mean_ratio_3m_12m",
  "acceleration_3m",
  "acceleration_12m",
  "volatility_3m",
  "volatility_12m",
  "drawdown",
```

```

    "monthly_change",
    "trend_strength",
    "acceleration_6m",
    "volatility_6m"
  )

task <- as_task_classif(features, target = "direction", id = "all_trends_raw")
task$select(features_to_use)

train_ids <- which(features$date < cutoff_date)
test_ids <- which(features$date >= cutoff_date & features$date < test_end)

resampling <- rsmp("custom")
resampling$instantiate(
  task,
  train_sets = list(train_ids),
  test_sets = list(test_ids)
)

learners = lrns(c("classif.featureless", "classif.log_reg", "classif.kknn", "classif.rpart", "classif.ranger"))

bmr_design <- benchmark_grid(tasks = task, learners = learners, resamplings = resampling)

bmr <- benchmark(design = bmr_design)

```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## Warning: package 'kknn' was built under R version 4.5.3
```

```

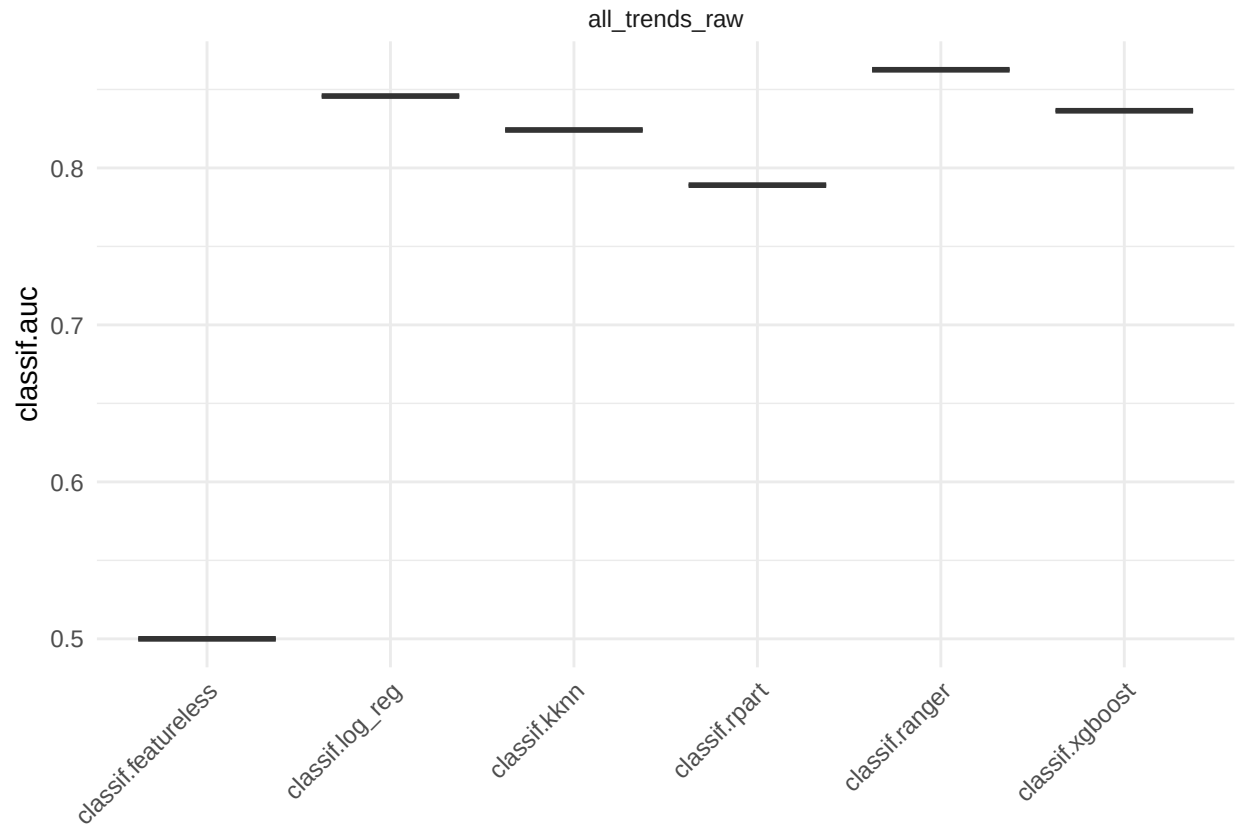
bmr$aggregate(measures = list(
  msr("classif.acc"),
  msr("classif.bacc"),
  msr("classif.auc"),
  msr("classif.bbrier"),
  msr("classif.precision"),
  msr("classif.recall")
))

```

| ##    | nr           | task_id        | learner_id          | resampling_id     | iters          | classif.acc |
|-------|--------------|----------------|---------------------|-------------------|----------------|-------------|
| ##    | <int>        | <char>         | <char>              | <char>            | <int>          | <num>       |
| ## 1: | 1            | all_trends_raw | classif.featureless | custom            | 1              | 0.6305066   |
| ## 2: | 2            | all_trends_raw | classif.log_reg     | custom            | 1              | 0.7615639   |
| ## 3: | 3            | all_trends_raw | classif.kknn        | custom            | 1              | 0.7351322   |
| ## 4: | 4            | all_trends_raw | classif.rpart       | custom            | 1              | 0.7560573   |
| ## 5: | 5            | all_trends_raw | classif.ranger      | custom            | 1              | 0.7593612   |
| ## 6: | 6            | all_trends_raw | classif.xgboost     | custom            | 1              | 0.7422907   |
| ##    | classif.bacc | classif.auc    | classif.bbrier      | classif.precision | classif.recall |             |
| ##    | <num>        | <num>          | <num>               | <num>             | <num>          |             |
| ## 1: | 0.5000000    | 0.5000000      | 0.2398442           | NaN               | 0.0000000      |             |
| ## 2: | 0.7584762    | 0.8458118      | 0.1624734           | 0.6557592         | 0.7466468      |             |
| ## 3: | 0.7449189    | 0.8241932      | 0.1859735           | 0.6104651         | 0.7824143      |             |
| ## 4: | 0.7454721    | 0.7890459      | 0.1806210           | 0.6587744         | 0.7049180      |             |

```
## 5:    0.7703024    0.8625411    0.1590258    0.6366822    0.8122206
## 6:    0.7555314    0.8364209    0.2038128    0.6154721    0.8062593
## Hidden columns: resample_result
```

```
autoplot(bmr, measure = msr("classif.auc"))
```



## Conclusion

This project asked a simple question: can Google search interest data anticipate which foods are about to grow in popularity? The answer is dependent on how the question is framed.

Predicting the exact future level of search interest proved to be a difficult problem. A naive model that simply assumed each keyword's recent search volume would continue unchanged explained a large share of the variance (R-squared = 0.662). Of the models tested, only XGboost clearly improved on this baseline (R-squared = 0.735). Ranger offered a marginal gain, and the simpler models performed worse than this naive model.

Reframing the initial problem as a binary question (will this keyword's search interest be higher or lower six months from now?), produced a substantially stronger and more consistent result. Against a naive baseline of always guessing the majority class (accuracy = 0.631, AUC = 0.500), the best-performing model (ranger) reached an accuracy of 0.757 and an AUC of 0.861, which is a significant improvement over every model tested. This suggests that while the exact magnitude of future search interest is hard to predict, the direction of change carries real, learnable signal.

Limitations:

There are a couple of limitations to this analysis. To start, Google search interest is a proxy for food popularity, rather than a pure measure of how much of that food is consumed. An increase in searches is not a direct connection to a food being consumed more. Also, I did not do any hyperparameter tuning; Ranger and XGBoost were run with default settings. Additionally, the analysis only uses U.S Google search data, so the results are only able to be generalized to Americans. Lastly, results are based on one time-based split (train through 2024, test on 2025–mid-2026). This could potentially give a noisier estimate of performance; a rolling-origin validation across multiple cutoff dates would give a more reliable sense of how stable the 0.86 AUC actually is across different time windows.

Future work:

Future work could focus on improving the model’s ability to identify emerging food trends by incorporating additional predictors. In particular, accounting for seasonality more explicitly could help distinguish recurring seasonal increases in search interest from genuine emerging trends.